Operating Systems

Assignment 5

Process Synchronisation

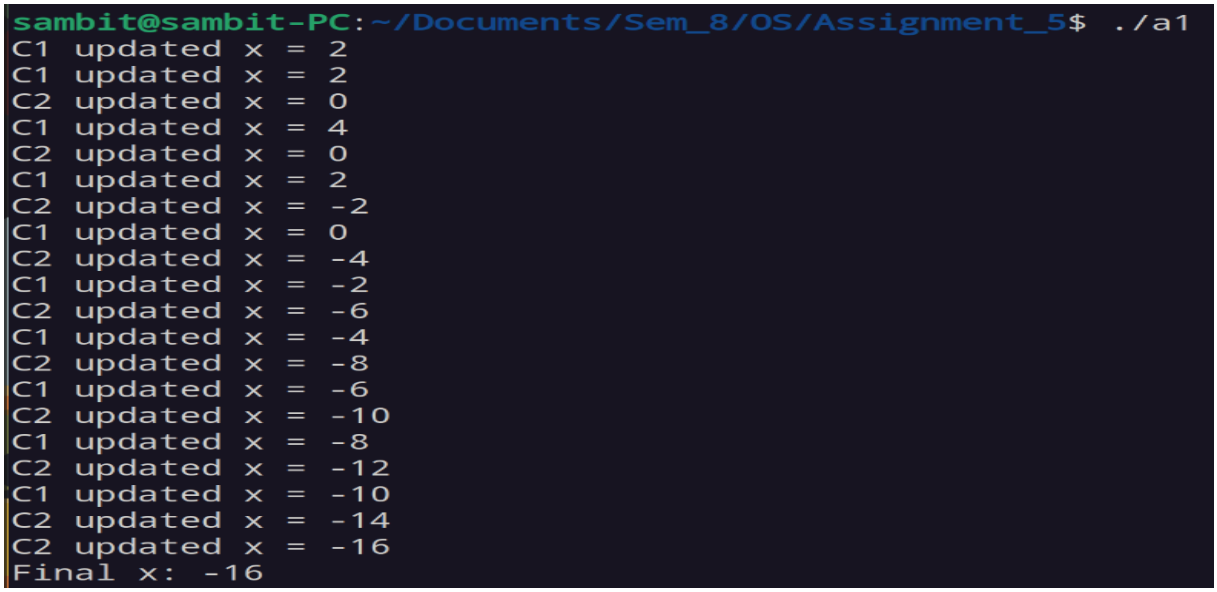
Sambit Sahoo (22277)

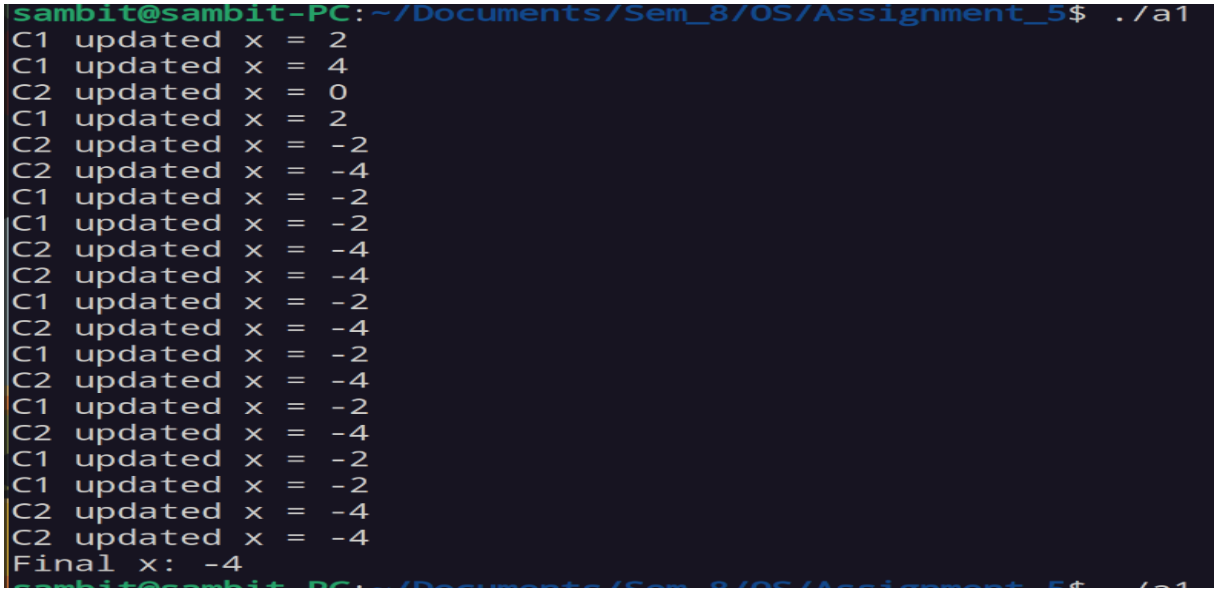
---

*My username is **sambit** (You can find it in all screenshots.)*

Q1. Consider a system with one parent process and two child processes, C1 and C2. There is a shared integer X initialized to 0. Process C1 increments X by two 10 times in a for loop. Meanwhile, process C2 decrements X by two 10 times in a for loop. After both C1 and C2 are finished, the parent process prints out the final value of X.

Outputs





```
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_5$ ./a1
C1 updated x = 2
C1 updated x = 4
C2 updated x = 0
C1 updated x = 2
C2 updated x = -2
C1 updated x = 0
C2 updated x = -4
C1 updated x = -2
C2 updated x = -6
C1 updated x = -4
C2 updated x = -8
C1 updated x = -6
C2 updated x = -10
C2 updated x = -12
C1 updated x = -8
C2 updated x = -14
C1 updated x = -12
C2 updated x = -16
C1 updated x = -14
C2 updated x = -18
Final x: -18
```

```
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_5$ ./a1
C1 updated x = 2
C1 updated x = 2
C2 updated x = 0
C1 updated x = 4
C2 updated x = 0
C1 updated x = 2
C2 updated x = -2
C1 updated x = 0
C2 updated x = -4
C1 updated x = -2
C2 updated x = -6
C2 updated x = -8
C1 updated x = -6
C2 updated x = -8
C1 updated x = -6
C2 updated x = -10
C1 updated x = -8
C2 updated x = -12
C1 updated x = -10
C2 updated x = -14
Final x: -14
```

Program

sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_5

```c
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/shm.h>
#include<sys/wait.h>

int main(){
	pid_t c, d;

	int shmid = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT | 0666);	//IPC_Private (only processes related to fork can utilize this)
	int *x = (int *) shmat(shmid, NULL, 0);				//Without this, we cannot access shared memory.
	*x = 0;

	c = fork();

	if (c != 0){
		d = fork();
	}

	if (c == 0){
		for (int i = 0; i < 10; i++){
			int temp = *x;
			temp = temp + 2;
			*x = temp;
			printf("C1 updated x = %d\n", *x);
			sleep(1);				// sleep between ith write and (i+1)th read
		}
	}
```

```
    else if (c != 0 && d == 0){
            for (int i = 0; i < 10; i++){
                int temp = *x;
                sleep(1);                           // sleep between ith read and (i+1)th write
                temp = temp - 2;
                *x = temp;
                printf("C2 updated x = %d\n", *x);
            }
    }

    else{
            wait(NULL);
            wait(NULL);
            printf("Final x: %d\n", *x);

            shmdt(x);
            shmctl(shmid, IPC_RMID, NULL);
    }

    return 0;
}
```

Explanation

- **Shared Memory:** A shared integer variable X is created using `shmget()` and attached using `shmat()`, allowing multiple processes to access the same memory.
- **Process Creation:** Two child processes are created using `fork()`:
  - $C1$: increments X by 2 (10 times)
  - $C2$: decrements X by 2 (10 times)
- **Race Condition Setup:** No synchronization mechanism (like semaphores) is used. Both processes directly access and modify the shared variable X.
- **Critical Section:** The operations:

$$X = X \pm 2$$

  are executed without mutual exclusion, leading to possible interleaving.
- **Inducing Interleaving:** `sleep()` calls are introduced at different points in $C1$ (kth write and k-1th read) and $C2$ (kth read and kth write) to force context switching and increase chances of race conditions.
- **Expected Behavior:** Ideally, the final value of X should be 0. However, due to race conditions, incorrect values may occur in some executions.
- **Observation:** The output may vary across runs, demonstrating the non-deterministic nature of concurrent execution without synchronization.

Q2. Add synchronization code based on semaphores to process C1 and C2 above so that race conditions are not possible. Use the call semget(), semop(), semctl() in linux to create and manage semaphore.

Output

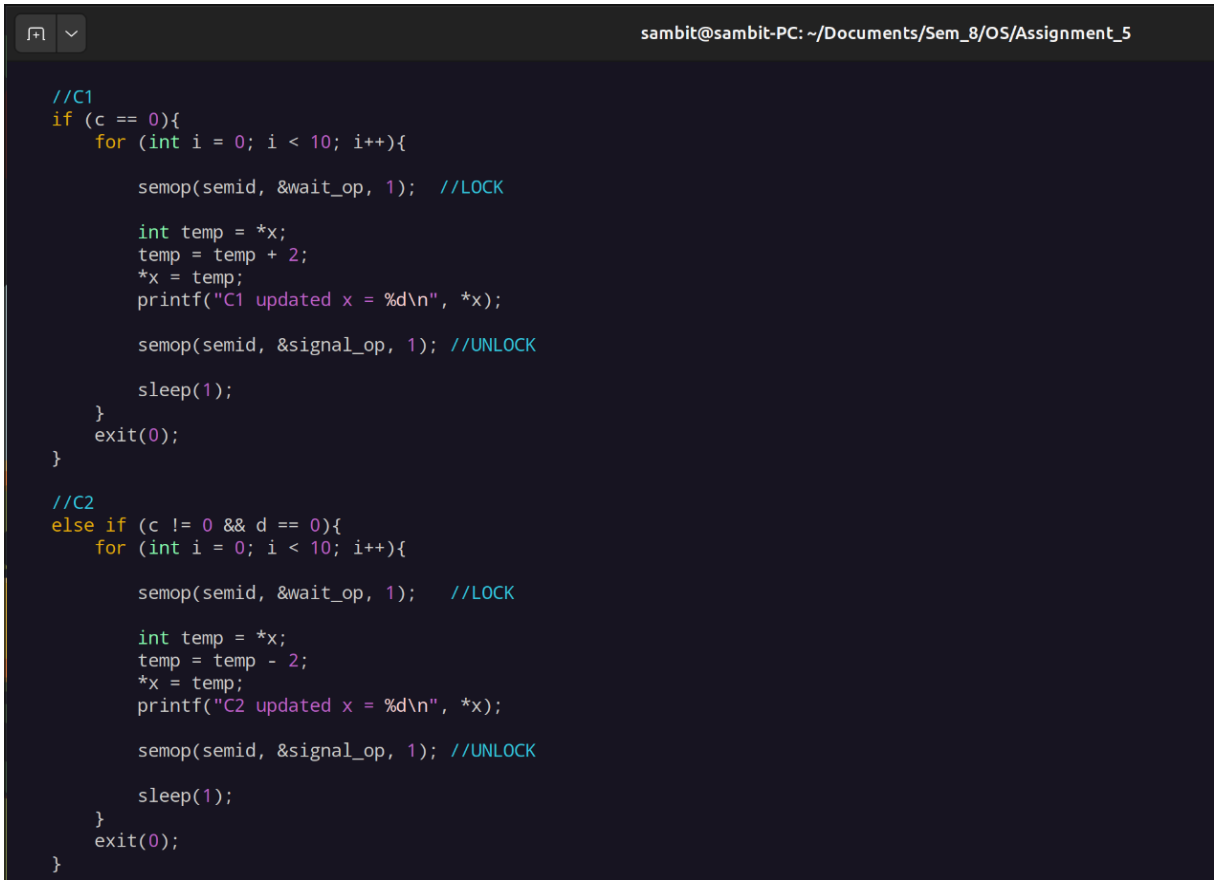
```
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_5$ ./a2
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
Final x: 0
```

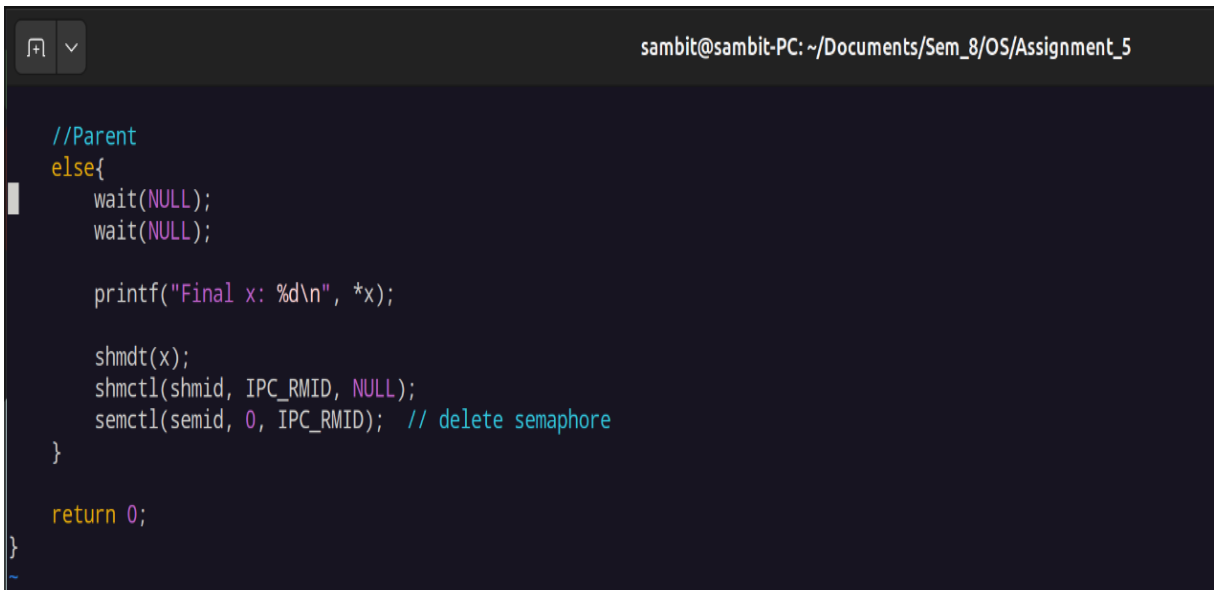
```
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_5$ ./a2
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
C1 updated x = 2
C2 updated x = 0
Final x: 0
```

Program

sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_5

```c
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/shm.h>
#include<sys/wait.h>
#include<sys/sem.h>

// union required for semctl
union semun {
    int val;
};

int main(){
    pid_t c, d;

    int shmid = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT | 0666);
    int *x = (int *) shmat(shmid, NULL, 0);
    *x = 0;

    //Semaphore creation (1 semaphore)
    int semid = semget(IPC_PRIVATE, 1, IPC_CREAT | 0666);

    union semun arg;
    arg.val = 1;  // initialize to 1 (mutex)
    semctl(semid, 0, SETVAL, arg);

    //Defined wait (P) and signal (V)
    struct sembuf wait_op = {0, -1, 0};    // decrement
    struct sembuf signal_op = {0, 1, 0};  // increment

    c = fork();

    if (c != 0){
        d = fork();
    }
```





Explanation

- **Introduction of Semaphore:** A binary semaphore is created using `semget()` to enforce mutual exclusion.
- **Initialization:** The semaphore is initialized to 1 using `semctl()`, representing an unlocked state (mutex).
- **Critical Section Protection:** The update operations on shared variable X are enclosed within semaphore operations:

  - `semop(..., -1)` : Wait (lock)
  - `semop(..., +1)` : Signal (unlock)

- **Elimination of Race Condition:** Only one process can access the critical section at a time, preventing simultaneous updates to X.
- **Deterministic Output:** Unlike Question 1, the final value of X is always 0 due to proper synchronization.
- **Semaphore Cleanup:** The semaphore is removed after execution using the `semctl()` system call with the `IPC_RMID` command, which deallocates the semaphore set from the system.

Q3.
In this program, you will write a program to solve the m-producer n-consumer problem, where $m, n \geq 1$.
You have a shared circular buffer that can hold 20 integers. Each producer process stores the numbers from 1 to 100 in the buffer one by one (in a `for` loop with 100 iterations) and then exits.
Each consumer process reads numbers from the buffer and adds them to a shared variable `SUM` (initialized to 0). Any consumer process can read any number in the buffer. The only constraint is that each number written by a producer must be read exactly once by some consumer.
However:

- A producer should not write when the buffer is full.
- A consumer should not read when the buffer is empty.

Write a program that:

- Creates the shared circular buffer and the shared variable `SUM` using `shm*()` system calls in Linux.
- You may create any additional shared variables as needed.
- Reads the values of m and n from the user.
- Forks m producer processes and n consumer processes.

The producer and consumer codes can be implemented as functions that are executed by the child processes.
All consumers must terminate only after all the data produced by all producers has been consumed. (Hint: Think about how a consumer determines that production is complete.)
After all producer and consumer processes finish execution, the parent process prints the value of `SUM`.
Test your program with the following cases:

- (a) $m = 1$, $n = 1$
- (b) $m = 1$, $n = 2$
- (c) $m = 2$, $n = 1$

- (d) $m = 2$, $n = 2$

Output

```
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_5$ gcc a3_final.c -o a3_final
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_5$ ./a3_final
Enter producers: 1
Enter consumers: 1

Final SUM = 5050
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_5$ ./a3_final
Enter producers: 1
Enter consumers: 2

Final SUM = 5050
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_5$ ./a3_final
Enter producers: 2
Enter consumers: 2

Final SUM = 10100
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_5$ ./a3_final
Enter producers: 2
Enter consumers: 1

Final SUM = 10100
```

Program

sambit@sambit-PC:

```c
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/shm.h>
#include<sys/wait.h>
#include<sys/sem.h>

#define SIZE 20

// required for semctl
union semun {
    int val;
};

int *buf, *sum, *in, *out;
int semid;

void P(int sem_num){                            // wait (P)
    struct sembuf op = {sem_num, -1, 0};
    semop(semid, &op, 1);
}

void V(int sem_num){                            // signal (V)
    struct sembuf op = {sem_num, 1, 0};
    semop(semid, &op, 1);
}

void producer(){
    for(int i = 1; i <= 100; i++){

        P(1); // empty--
        P(0); // mutex

        buf[*in] = i;
        *in = (*in + 1) % SIZE;

        V(0); // mutex
        V(2); // full++
    }
    exit(0);
}
```

```
sambit@sambit-PC:
```

```
void consumer(){
    while(1){

        P(2); // full--
        P(0); // mutex

        int item = buf[*out];
        *out = (*out + 1) % SIZE;

        V(0); // mutex
        V(1); // empty++

        if(item == -1)
            break;

        *sum += item;
    }
    exit(0);
}

int main(){

    int shmid[4];

    //shared buffer
    shmid[0] = shmget(IPC_PRIVATE, SIZE*sizeof(int), IPC_CREAT | 0666);
    buf = (int*) shmat(shmid[0], NULL, 0);

    //sum
    shmid[1] = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT | 0666);
    sum = (int*) shmat(shmid[1], NULL, 0);
    *sum = 0;

    //in index
    shmid[2] = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT | 0666);
    in = (int*) shmat(shmid[2], NULL, 0);
    *in = 0;

    //out index
    shmid[3] = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT | 0666);
    out = (int*) shmat(shmid[3], NULL, 0);
    *out = 0;
```

```
    semid = semget(IPC_PRIVATE, 3, IPC_CREAT | 0666);              //semaphores: 0=mutex, 1=empty, 2=full

    union semun arg;

    arg.val = 1;
    semctl(semid, 0, SETVAL, arg); // mutex

    arg.val = SIZE;
    semctl(semid, 1, SETVAL, arg); // empty

    arg.val = 0;
    semctl(semid, 2, SETVAL, arg); // full

    int m, n;
    printf("Enter producers: ");
    scanf("%d", &m);
    printf("Enter consumers: ");
    scanf("%d", &n);

    pid_t pid;

    for(int i = 0; i < m; i++){
        if((pid = fork()) == 0){
            producer();
        }
    }

    for(int i = 0; i < n; i++){
        if((pid = fork()) == 0){
            consumer();
        }
    }

    for(int i = 0; i < m; i++)
        wait(NULL);
```

```
    for(int i = 0; i < n; i++){

        P(1); // empty
        P(0); // mutex

        buf[*in] = -1;
        *in = (*in + 1) % SIZE;

        V(0);
        V(2); // full
    }

    for(int i = 0; i < n; i++)
        wait(NULL);

    printf("\nFinal SUM = %d\n", *sum);

    shmdt(buf); shmdt(sum); shmdt(in); shmdt(out);

    for(int i = 0; i < 4; i++)
        shmctl(shmid[i], IPC_RMID, NULL);

    semctl(semid, 0, IPC_RMID);

    return 0;
}
```

Explanation

- **Shared Memory Setup:** Shared memory segments are created using `shmget()` and attached using `shmat()` for:
  - Circular buffer of size 20
  - Shared variable `SUM`
  - Indices `in` (producer) and `out` (consumer)
- **Circular Buffer:** A fixed-size buffer of 20 integers is used with modulo arithmetic:

  `index = (index + 1) % SIZE`

  to implement circular behavior.
- **Semaphore Initialization:** Three semaphores are created using `semget()`:
  - `mutex = 1` (mutual exclusion)
  - `empty = 20` (available slots)
  - `full = 0` (occupied slots)
- **Producer Operation:** Each producer inserts integers from 1 to 100 into the buffer:
  - Wait on `empty` and `mutex`
  - Write to buffer and update `in`
  - Signal `mutex` and `full`

- **Consumer Operation:** Each consumer removes items from the buffer and adds them to `SUM`:
  - Wait on `full` and `mutex`
  - Read from buffer and update `out`
  - Signal `mutex` and `empty`
- **Termination Mechanism:** After all producers finish, the parent inserts `-1` (termination signals) into the buffer equal to the number of consumers. Each consumer exits upon reading `-1`.
- **Process Creation:** The parent process forks m producer processes and n consumer processes using `fork()`.
- **Synchronization:** Semaphore operations `semop()` implement wait (P) and signal (V) to ensure correct synchronization and avoid race conditions.
- **Final Output and Cleanup:** After all processes terminate, the parent prints the final value of `SUM` and releases all shared memory and semaphore resources using `shmdt()`, `shmctl()`, and `semctl()`.